

Generating Setup and Hold Timing Reports for GCD (Greatest Common Divisor) Using Opensta

INTRODUCTION:

In this lab, students will learn how to perform static timing analysis on a synthesized GCD using OpenSTA. They will practice importing the gate-level netlist from Yosys, loading timing constraints, linking standard cell libraries, and generating detailed timing reports. This exercise demonstrates how timing is verified for hardware-efficient logic at the signoff level.

APPLICATION:

Digital GCD modules are used in cryptography algorithms, digital signal processing, coding theory, error correction, and computational arithmetic engines. They also serve as essential components for hardware accelerators, coprocessors, and digital systems requiring efficient mathematical operations, such as modular reduction and rational number simplification.

OBJECTIVE:

After completing this lab, you will be able to:

- Import a gate-level netlist and standard cell library into OpenSTA (OSU018).
- Load and apply timing constraints (e.g., clock, input/output delays) using SDC format.
- Perform static timing analysis to check setup and hold margins across the design.
- Understand timing reports, including slack values, critical paths, and violation detection.
- Interpret timing analysis results to guide optimization of digital designs for timing closure.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <project_name>`
- 1.3: Create directory for project: `mkdir gcd`
- 1.4: Open project directory: `cd gcd`

Step 2: Create Design file

2.1: Open gvim editor for design with command: gvim gcd.v

RTL Design Code:

```
module gcd (  
    input clk,  
    input rst,  
    input start,  
    input [31:0] A,  
    input [31:0] B,  
    output reg done,  
    output reg [31:0] result  
);  
  
    // FSM states  
    localparam IDLE = 2'd0,  
                RUN  = 2'd1,  
                DONE = 2'd2;  
  
    reg [1:0] state;  
    reg [31:0] a_reg, b_reg;  
  
    always @(posedge clk or posedge rst) begin  
        if (rst) begin  
            // Reset all signals  
            state <= IDLE;  
            done <= 0;  
            result <= 0;  
            a_reg <= 0;  
            b_reg <= 0;  
        end else begin  
            case (state)  
                IDLE: begin  
                    done <= 0;  
                    if (start) begin  
                        a_reg <= A;  
                        b_reg <= B;  
                        state <= RUN;  
                    end  
                end  
  
                RUN: begin  
                    if (b_reg == 0) begin  
                        result <= a_reg;  
                        done <= 1;  
                    end  
                end  
            end  
        end  
    end
```

```
        state <= DONE;
    end else if (a_reg > b_reg) begin
        a_reg <= a_reg - b_reg;
    end else begin
        b_reg <= b_reg - a_reg;
    end
end
end

DONE: begin
    // Wait for next start
    if (!start) begin
        state <= IDLE;
        done <= 0;
    end
end

default: begin
    // Safety fallback for invalid states
    state <= IDLE;
    done <= 0;
    result <= 0;
end
endcase
end
end
endmodule
```

2.2: save the file using command- :wq

Step 3: Create .sdc file

3.1: Open gvim editor for the SDC with command: gvim gcd.sdc

For 10ns:

```
create_clock -period 10 -name clk [get_ports clk]
set_input_delay 5 -min -rise [get_ports clk] -clock clk
set_input_delay 5 -min -rise [get_ports {input_dat rst enable}] -clock
clk
set_input_transition 15 -min -fall [get_ports {input_dat rst enable}]
-clock clk
set_input_transition 15 -max -fall [get_ports clk] -clock clk
set_load -pin_load 4 [get_ports {output_data sampleT}]
set_output_delay 5 -max -rise [get_ports {output_data sampleT}] -clock
clk
```

Note: For 5ns, change the clock period to 5ns in sdc.

Step 4: Static Timing Analysis for the design with OpenSTA

Invoke OpenSTA in the terminal using the command: `sta`

4:1 Use the following commands:

```
# Loads the liberty to osu018 standard cell library
i). read_liberty osu018_stdcells.lib

# Reads your synthesized gate-level Verilog netlist
ii). read_verilog gcd_synth.v

# Links the loaded netlist and library to the top module name
iii). link_design gcd

# Loads your timing constraints from an SDC file
iv). read_sdc gcd.sdc

# Runs timing analysis to check all timing constraints and generates a report.
v). report_checks
```

OBSERVATION:

- Detect setup and hold timing violations
- Examine input and output delay constraints
- Review total negative slack and worst slack reports

OUTPUT:

Reports:

For 10ns:

➔ for setup

```
OpenSTA> report_checks
Startpoint: rst (input port clocked by clk)
Endpoint: _1622_ (recovery check against rising-edge clock clk)
Path Group: **async_default**
Path Type: max
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.50	0.50 v	input external delay
0.00	0.50 v	rst (in)
3.49	3.99 ^	_1523_/Y (INVX1)
0.00	3.99 ^	_1622_/R (DFFSR)
	3.99	data arrival time
10.00	10.00	clock clk (rise edge)
0.00	10.00	clock network delay (ideal)
0.00	10.00	clock reconvergence pessimism
	10.00 ^	_1622_/CLK (DFFSR)
-0.57	9.43	library recovery time
	9.43	data required time
	9.43	data required time
	-3.99	data arrival time
	5.44	slack (MET)

```
Startpoint: _1623_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _1624_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	^ _1623_/CLK (DFFSR)
1.66	1.66	^ _1623_/Q (DFFSR)
2.77	4.43	v _1050_/Y (MUX2X1)
0.40	4.83	^ _1051_/Y (AOI22X1)
0.06	4.89	v _1052_/Y (OAI21X1)
0.00	4.89	v _1624_/D (DFFSR)
	4.89	data arrival time
10.00	10.00	clock clk (rise edge)
0.00	10.00	clock network delay (ideal)
0.00	10.00	clock reconvergence pessimism
	10.00	^ _1624_/CLK (DFFSR)
-0.10	9.90	library setup time
	9.90	data required time
	9.90	data required time
	-4.89	data arrival time
	5.02	slack (MET)

→ [for hold](#)

```
OpenSTA> report_checks -path_delay min
Startpoint: rst (input port clocked by clk)
Endpoint: _1621_ (removal check against rising-edge clock clk)
Path Group: **asyn_default**
Path Type: min
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
5.00	5.00	v input external delay
0.00	5.00	v rst (in)
1.78	6.78	^ _1522_/Y (INVX1)
0.00	6.78	^ _1621_/S (DFFSR)
	6.78	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
	0.00	^ _1621_/CLK (DFFSR)
0.08	0.08	library removal time
	0.08	data required time
	0.08	data required time
	-6.78	data arrival time
	6.70	slack (MET)

```
Startpoint: _1688_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _1688_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	^ _1688_/CLK (DFFSR)
0.23	0.23	^ _1688_/Q (DFFSR)
0.05	0.27	v _1237_/Y (NAND2X1)
0.05	0.32	^ _1238_/Y (OAI21X1)
0.00	0.32	^ _1688_/D (DFFSR)
	0.32	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
	0.00	^ _1688_/CLK (DFFSR)
0.01	0.01	library hold time
	0.01	data required time
	0.01	data required time
	-0.32	data arrival time
	0.32	slack (MET)

→ [for setup](#)

```
OpenSTA> report_checks
Startpoint: rst (input port clocked by clk)
Endpoint: _1622_ (recovery check against rising-edge clock clk)
Path Group: **async_default**
Path Type: max
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.50	0.50 v	input external delay
0.00	0.50 v	rst (in)
3.49	3.99 ^	_1523_/Y (INVX1)
0.00	3.99 ^	_1622_/R (DFFSR)
	3.99	data arrival time
5.00	5.00	clock clk (rise edge)
0.00	5.00	clock network delay (ideal)
0.00	5.00	clock reconvergence pessimism
	5.00 ^	_1622_/CLK (DFFSR)
-0.57	4.43	library recovery time
	4.43	data required time
	4.43	data required time
	-3.99	data arrival time
	0.44	slack (MET)

```
Startpoint: _1623_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _1624_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00 ^	_1623_/CLK (DFFSR)
1.66	1.66 ^	_1623_/Q (DFFSR)
2.77	4.43 v	_1050_/Y (MUX2X1)
0.40	4.83 ^	_1051_/Y (AOI22X1)
0.06	4.89 v	_1052_/Y (OAI21X1)
0.00	4.89 v	_1624_/D (DFFSR)
	4.89	data arrival time
5.00	5.00	clock clk (rise edge)
0.00	5.00	clock network delay (ideal)
0.00	5.00	clock reconvergence pessimism
	5.00 ^	_1624_/CLK (DFFSR)
-0.10	4.90	library setup time
	4.90	data required time
	4.90	data required time
	-4.89	data arrival time
	0.02	slack (MET)

→ [for hold](#)

```
OpenSTA> report_checks -path_delay min
Startpoint: rst (input port clocked by clk)
Endpoint: _1621_ (removal check against rising-edge clock clk)
Path Group: **async_default**
Path Type: min
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
5.00	5.00 v	input external delay
0.00	5.00 v	rst (in)
1.78	6.78 ^	_1522_/Y (INVX1)
0.00	6.78 ^	_1621_/S (DFFSR)
	6.78	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
	0.00 ^	_1621_/CLK (DFFSR)
0.08	0.08	library removal time
	0.08	data required time
	0.08	data required time
	-6.78	data arrival time
	6.70	slack (MET)


```

Startpoint: _1688_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _1688_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min
    
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	^ _1688_/CLK (DFFSR)
0.23	0.23	^ _1688_/Q (DFFSR)
0.05	0.27	v _1237_/Y (NAND2X1)
0.05	0.32	^ _1238_/Y (OAI21X1)
0.00	0.32	^ _1688_/D (DFFSR)
	0.32	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
0.00	0.00	^ _1688_/CLK (DFFSR)
0.01	0.01	library hold time
	0.01	data required time
	-0.32	data arrival time
	0.32	slack (MET)

COMPARISION:

Timing	Setup		Hold	
	Path 1	Path 2	Path 1	Path 2
10ns	5.44	5.04	6.70	0.32
5ns	0.44	0.02	6.70	0.32

Note: Timing can be changed by changing constraints in sdc.

CONCLUSION:

OpenSTA verifies the timing of the synthesized GCD design using gate-level netlist and timing constraints. The analysis confirms that all critical paths meet setup and hold requirements, with positive slack values. No timing violations are detected, ensuring reliable operation at the specified clock frequency. The clock, reset, and counter increment logic show correct timing relationships in the reports. Overall, OpenSTA validates that the hardware counter will function accurately in real digital systems.